

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

**MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR**

Université de Yaoundé I

Institut Saint Jean**REPUBLIC OF CAMEROON**

Peace-Work-Fatherland

MINISTRY OF HIGHER EDUCATION

University of Yaounde I

Institut Saint Jean

CAHIER DE CONCEPTION

CONCEPTION, DEVELOPPEMENT ET DEPLOIEMENT D'UN WEB-BASED LINUX NAMESPACE ISOLATION MANAGER

Gestionnaire d'isolation de processus Linux via interface web

Rédigé et présenté par :

➔ *TCHINDA Florelle Ecladore*

➔ *PAYONG Brice Valery*

➔ *ZOUA HOULI Abraham*

Sous la supervision de M. NGUIMBUS Emmanuel

Année Académique

2025-2026

ÉTUDE DU PROJET

Nom du projet	Conception, développement et déploiement d'un Web-based Linux Namespace Isolation Manager
Version du document	2 ^e version
Date de création	13/06/2026
Dernière mise à jour	07/07/2026
Responsables du projet	➔ TCHINDA Florelle Ecladore ➔ PAYONG Brice Valery ➔ ZOUA HOULI Abraham
Validé par	M. NGUIMBUS Emmanuel

SOMMAIRE

Sommaire.....	2
1. INTRODUCTION ET CONTEXTE.....	3
1.1 Rôle de ce document.....	3
1.2 Ce que ce document modélise.....	3
2. ARCHITECTURE DE L'APPLICATION.....	3
2.1 Une architecture en couches, simplifiée.....	3
2.2 Les composants et leurs canaux de communication.....	4
2.3 Description des composants.....	5
2.4 Multi-utilisateur et sécurité système.....	5
Principe multi-utilisateur.....	5
Sécurité du compte système.....	6
3. DIAGRAMME DE CLASSES.....	6
3.1 Vue d'ensemble.....	6
3.2 Détail des classes.....	7
Utilisateur.....	7
Sandbox.....	7
Commande.....	7
StatutSandbox (énumération).....	7
4. DIAGRAMME DE SEQUENCE.....	7
4.1 DS0 — S'authentifier.....	7
4.2 DS1 — Créer un sandbox.....	8
4.3 DS2 — Lister les sandboxes.....	9
4.4 DS3 — Exécuter une commande.....	9
4.5 DS4 — Rappeler une commande précédente.....	10
4.6 DS5 — Supprimer une Sandbox.....	10
5. SYNTHÈSE ET PROCHAINES ÉTAPES.....	11
5.1 Ce que la conception a permis de valider.....	11
5.2 Prochaines étapes.....	11

1. INTRODUCTION ET CONTEXTE

1.1 Rôle de ce document

Ce cahier de conception traduit les spécifications fonctionnelles issues de l'analyse en structures techniques précises : classes, relations entre objets, séquences d'échanges entre composants et architecture applicative globale. Il constitue le pont entre le « quoi » (les besoins fonctionnels) et le « comment » (l'implémentation), et sert de référence pour la phase de développement à venir.

1.2 Ce que ce document modélise

- L'architecture en couches de l'application — la manière dont les grands composants du système sont organisés et communiquent entre eux ;
- La structure des données — le diagramme de classes, qui décrit les objets manipulés par l'application et leurs relations ;
- Les échanges entre composants — six diagrammes de séquence (DS0 à DS5), qui détaillent le déroulement pas à pas des principaux cas d'usage.

Au total, ce cahier rassemble quatre artéfacts de conception : l'architecture 3-tier (vision système), le diagramme de classes (structure statique), et les six diagrammes de séquence (comportement dynamique).

2. ARCHITECTURE DE L'APPLICATION

2.1 Une architecture en couches, simplifiée

L'application est bâtie sur le modèle classique à trois couches (« 3-tier ») : chaque couche a une responsabilité unique, et ne dialogue qu'avec la couche directement voisine. Ce cloisonnement rend le système plus facile à comprendre, à faire évoluer et à corriger, puisqu'une modification dans une couche n'affecte pas directement les autres.

- La couche présentation (Frontend) — ce que l'utilisateur voit et manipule dans son navigateur : formulaire de connexion, tableau de bord, gestion des sandboxes, terminal web.
- La couche logique métier (Backend) — le cerveau de l'application, qui reçoit les demandes du Frontend, prend les décisions et orchestre le reste du système.
- La couche données — l'endroit où l'information est conservée et où les actions concrètes sont réellement exécutées.

La particularité de ce projet se situe précisément dans cette troisième couche : elle se compose de deux éléments distincts, qui travaillent en parallèle plutôt qu'un seul :

- **La Base de Données** conserve les informations administratives qui doivent survivre à un redémarrage du serveur : comptes utilisateurs, sessions actives, sandboxes existantes et historique des commandes.
- **Le Système d'Exploitation Hôte** est la couche d'exécution réelle : c'est lui qui crée physiquement les environnements isolés (les sandboxes) et y exécute les commandes de l'utilisateur.

Autrement dit, la Base de Données retient la mémoire de ce qui existe, pendant que le Système d'Exploitation Hôte fait exister ces choses concrètement sur la machine. Les deux sont indispensables et complémentaires : sans la base, l'application oublierait tout au redémarrage ; sans le système hôte, il n'y aurait aucune sandbox à gérer.

Enfin, il est important de préciser ce que cette architecture n'est pas une architecture microservices. Il n'existe qu'un seul Backend monolithique, qui centralise toute la logique applicative — authentification, gestion des sandboxes, terminal, historique — au lieu de la répartir entre plusieurs services indépendants déployés séparément. Ce choix simplifie le déploiement et la maintenance pour un projet de cette envergure.

Couche	Composant(s)	Responsabilité
Présentation	Frontend	Afficher l'interface et transmettre les actions de l'utilisateur
Logique métier	Backend	Vérifier, décider, orchestrer l'ensemble du système
Données (à double visage)	Base de Données Système d'Exploitation Hôte	Conserver l'information persistante Exécuter réellement les sandboxes

Tableau 1 — Les trois couches de l'architecture et leurs composants

2.2 Les composants et leurs canaux de communication

Le schéma ci-dessous synthétise les composants applicatifs et la nature des échanges qui les relient : les navigateurs des utilisateurs chargent et exécutent le Frontend ; celui-ci dialogue avec le Backend via HTTPS et WebSocket (pour le terminal interactif) ; le Backend, exécuté sous un compte Linux dédié non-root, envoie des requêtes SQL à la Base de Données et transmet ses demandes d'isolation et d'exécution au Système d'Exploitation Hôte, qui gère à son tour chaque sandbox comme un processus bash emprisonné indépendant.

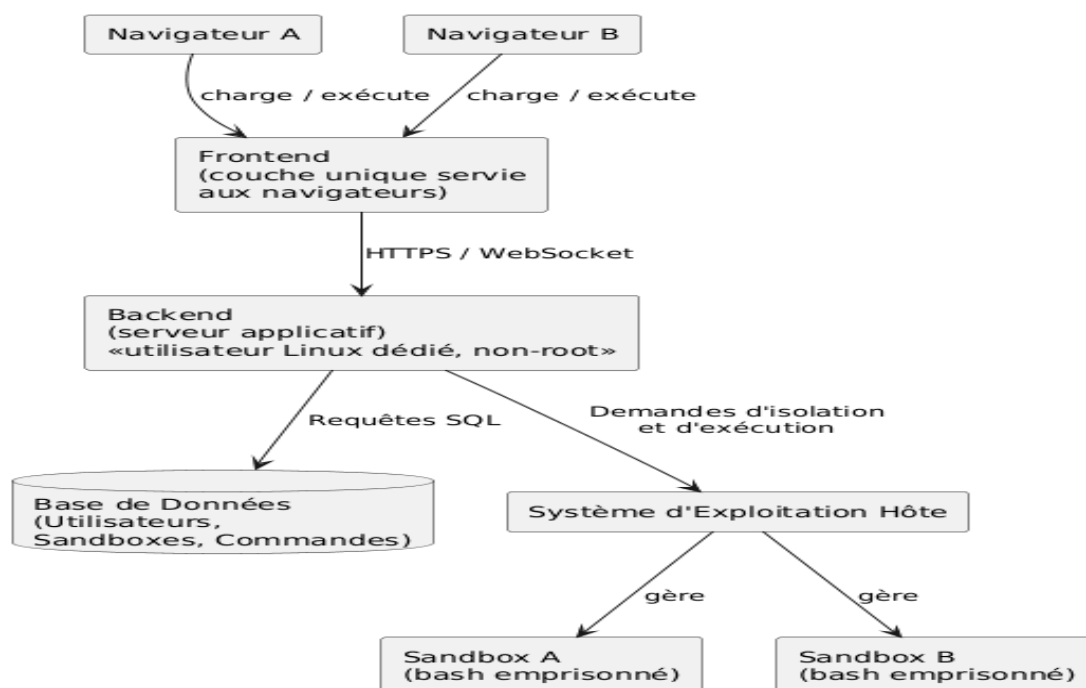


Figure 1 — Architecture de l'application

2.3 Description des composants

Composant	Rôle
Frontend	Couche développée et maîtrisée par l'équipe projet. Elle affiche le formulaire de connexion, le tableau de bord des sandboxes et le terminal web interactif. Elle ne prend aucune décision métier : elle capture les actions de l'utilisateur, les transmet au Backend, puis affiche la réponse reçue. Elle ne communique jamais directement avec la Base de Données ni avec le Système d'Exploitation Hôte.
Backend	Cœur logique de l'application, exécuté sous un utilisateur Linux dédié et non-root. Il reçoit toutes les requêtes du Frontend, vérifie les autorisations, orchestre la création et la destruction des sandboxes, transmet les commandes au Système d'Exploitation Hôte, et lit ou écrit dans la Base de Données. C'est lui qui filtre systématiquement chaque requête par l'identifiant de l'utilisateur connecté (proprietaireId).
Base de Données	Stockage durable des informations devant survivre à un redémarrage du serveur : comptes utilisateurs connus de l'application, sessions actives, sandboxes existantes avec leur propriétaire, et historique des commandes exécutées.
Système d'Exploitation Hôte	Couche d'exécution réelle, sur laquelle tout repose. C'est elle qui authentifie les comptes Linux réels, crée les bulles d'isolement (namespaces), démarre les processus bash emprisonnés, exécute les commandes, et libère les ressources à la destruction d'une sandbox.
Sandboxes (A et B)	Environnements isolés eux-mêmes, gérés directement par le Système d'Exploitation Hôte. Chacun dispose de ses propres namespaces — PID, UTS, NET, MNT, USER — totalement indépendants les uns des autres.

Tableau 2 — Description des composants de l'architecture

2.4 Multi-utilisateur et sécurité système

Principe multi-utilisateur

Chaque utilisateur authentifié obtient un identifiant propre. Chaque sandbox créée est rattachée à ce propriétaire (proprietaireId). À chaque requête, le Backend filtre systématiquement les données par cet identifiant : aucun utilisateur ne peut voir, manipuler ou détruire les sandboxes d'un autre, même en agissant au même moment.

Sécurité du compte système

Le Backend ne tourne jamais sous le compte root de la machine hôte. Il s'exécute sous un utilisateur Linux dédié, créé spécifiquement pour l'application, et ne disposant que des droits strictement nécessaires à la création de namespaces non privilégiés. Ce choix limite la surface d'attaque : une faille dans le Backend ne donnerait pas un accès administrateur complet à la machine hôte.

3. DIAGRAMME DE CLASSES

3.1 Vue d'ensemble

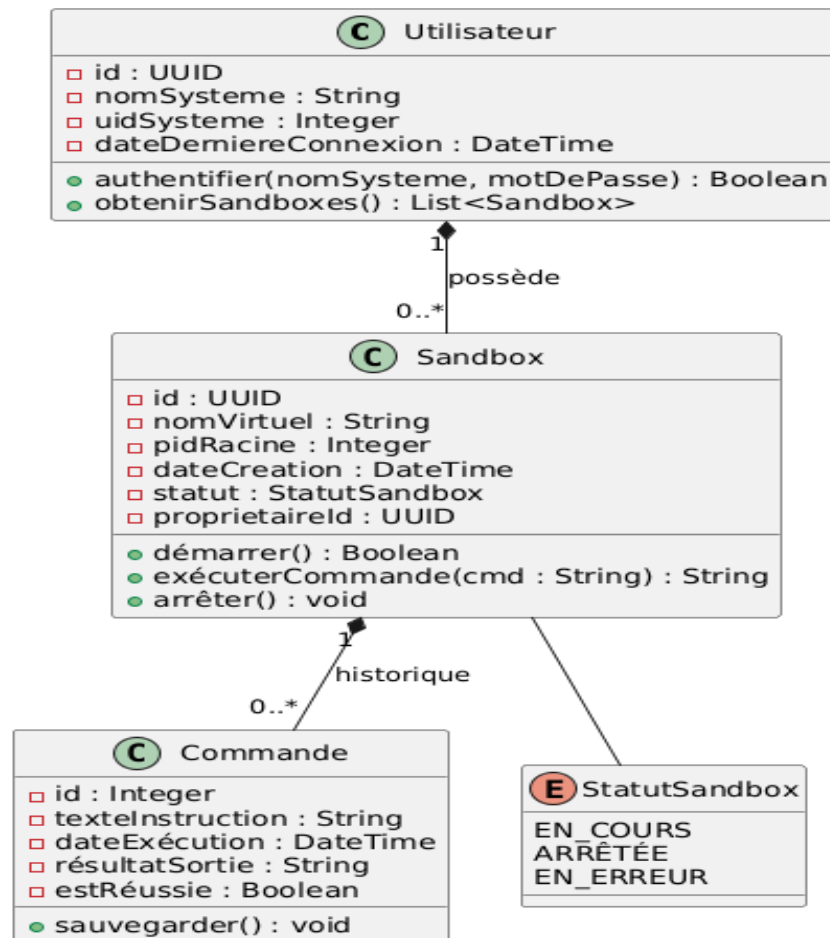


Figure 2 — Diagramme de classes

La structure statique de l'application repose sur trois classes principales et une énumération. Un Utilisateur possède zéro, une ou plusieurs Sandbox (relation « possède », cardinalité 1 à 0..*); chaque Sandbox conserve l'historique des Commande qui y ont été exécutées (relation « historique », cardinalité 1 à 0..*); le statut d'une Sandbox est décrit par l'énumération StatutSandbox.

3.2 Détail des classes

Utilisateur

Référence un compte Linux existant sur la machine hôte. Elle ne stocke jamais de mot de passe : l'authentification réelle est déléguée au système d'exploitation.

- **Attributs** : id (UUID) · nomSysteme (String) · uidSysteme (Integer) · dateDerniereConnexion (DateTime)
- **Méthodes** : authentifier(nomSysteme, motDePasse) : Boolean — vérifie les identifiants auprès du système hôte ; obtenirSandboxes() : List<Sandbox> — retourne les sandboxes appartenant à cet utilisateur.

Sandbox

Environnement isolé, rattaché à un utilisateur précis via `proprietaireId`.

- **Attributs** : `id` (UUID) · `nomVirtuel` (String) · `pidRacine` (Integer) · `dateCreation` (DateTime) · `statut` (StatutSandbox) · `proprietaireId` (UUID)
- **Méthodes** : `démarrer()` : Boolean — initialise l'espace isolé et bascule le statut à `EN_COURS` ; `exécuterCommande(cmd)` : String — transmet une instruction au bash emprisonné et retourne le résultat ; `arrêter()` : void — termine le processus racine et libère les ressources.

Commande

Trace persistée d'une instruction exécutée dans une sandbox.

- **Attributs** : `id` (Integer) · `texteInstruction` (String) · `dateExécution` (DateTime) · `résultatSortie` (String) · `estRéussie` (Boolean)
- **Méthodes** : `sauvegarder()` : void — insère l'enregistrement en base de données, en l'associant à l'identifiant de sa sandbox d'origine.

StatutSandbox (énumération)

Décrit l'état courant d'une sandbox à un instant donné : `EN_COURS` (active et utilisable), `ARRÊTÉE` (détruite ou stoppée), `EN_ERREUR` (échec de création ou d'exécution).

4. DIAGRAMME DE SEQUENCE

Les six diagrammes de séquence ci-après détaillent le déroulement pas à pas des principaux cas d'usage de l'application, depuis l'action de l'utilisateur dans son navigateur jusqu'à la réponse renvoyée par le système. Chaque diagramme distingue le chemin nominal (succès) des chemins alternatifs (échec, cas limites), représentés par des blocs conditionnels « alt ».

4.1 DS0 — S'authentifier

Ce scénario décrit la vérification de l'identité d'un utilisateur avant l'accès au tableau de bord. L'authentification réelle est déléguée au Système d'Exploitation Hôte, qui est seul juge des identifiants Linux ; le Backend ne fait que relayer la demande et exploiter le résultat.

Participants : Utilisateur Web · Frontend · Backend · Système d'Exploitation Hôte · Base de Données

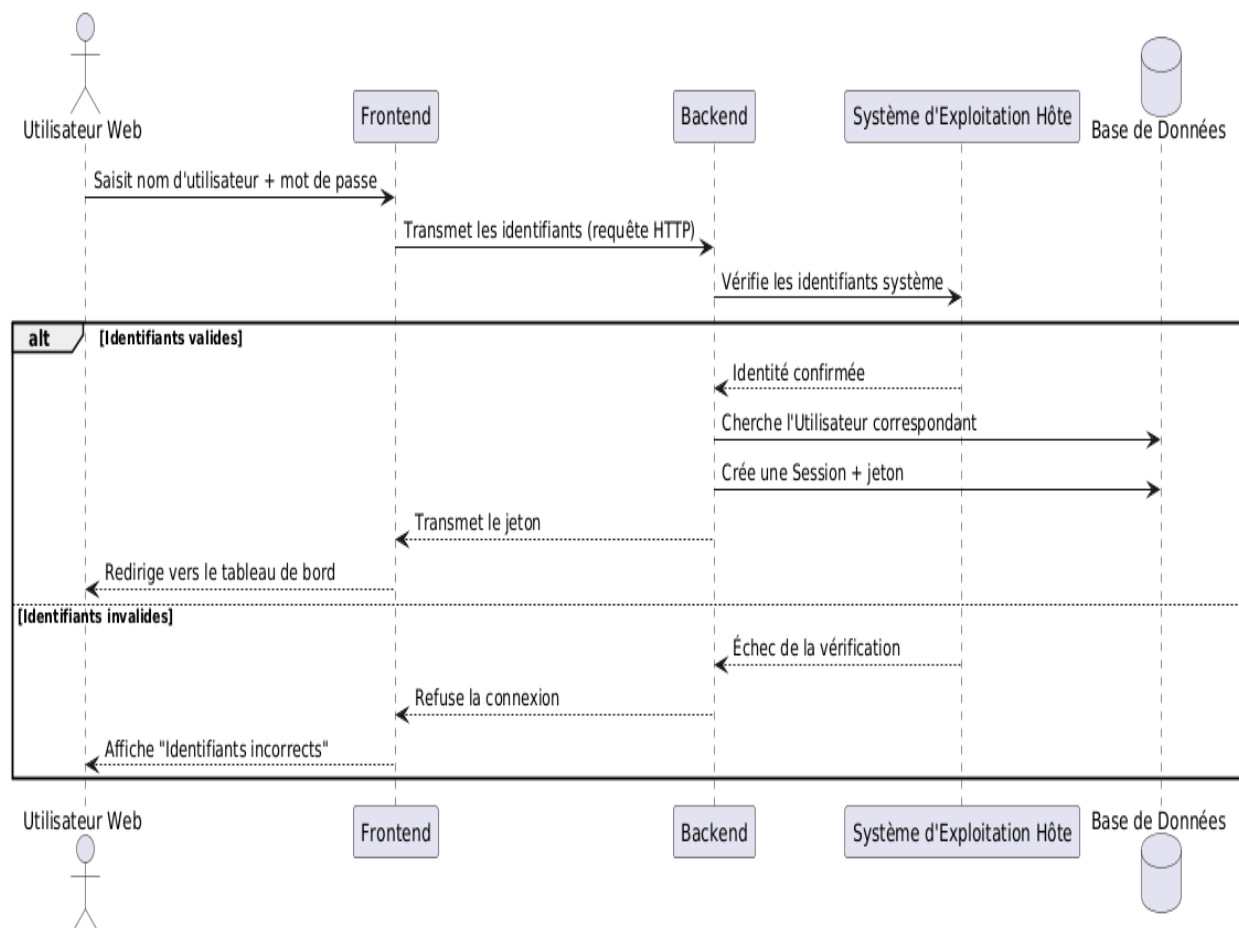


Figure 3 — DS0 S'authentifier

4.2 DS1 — Créer un sandbox

Ce scénario décrit la création d'un nouvel environnement isolé. Trois issues sont possibles : succès complet, succès partiel avec nettoyage automatique (la bulle est créée mais le terminal ne démarre pas), ou échec dès la création de la bulle d'isolement.

Participants : Utilisateur Web · Frontend · Backend · Système d'Exploitation Hôte · Base de Données

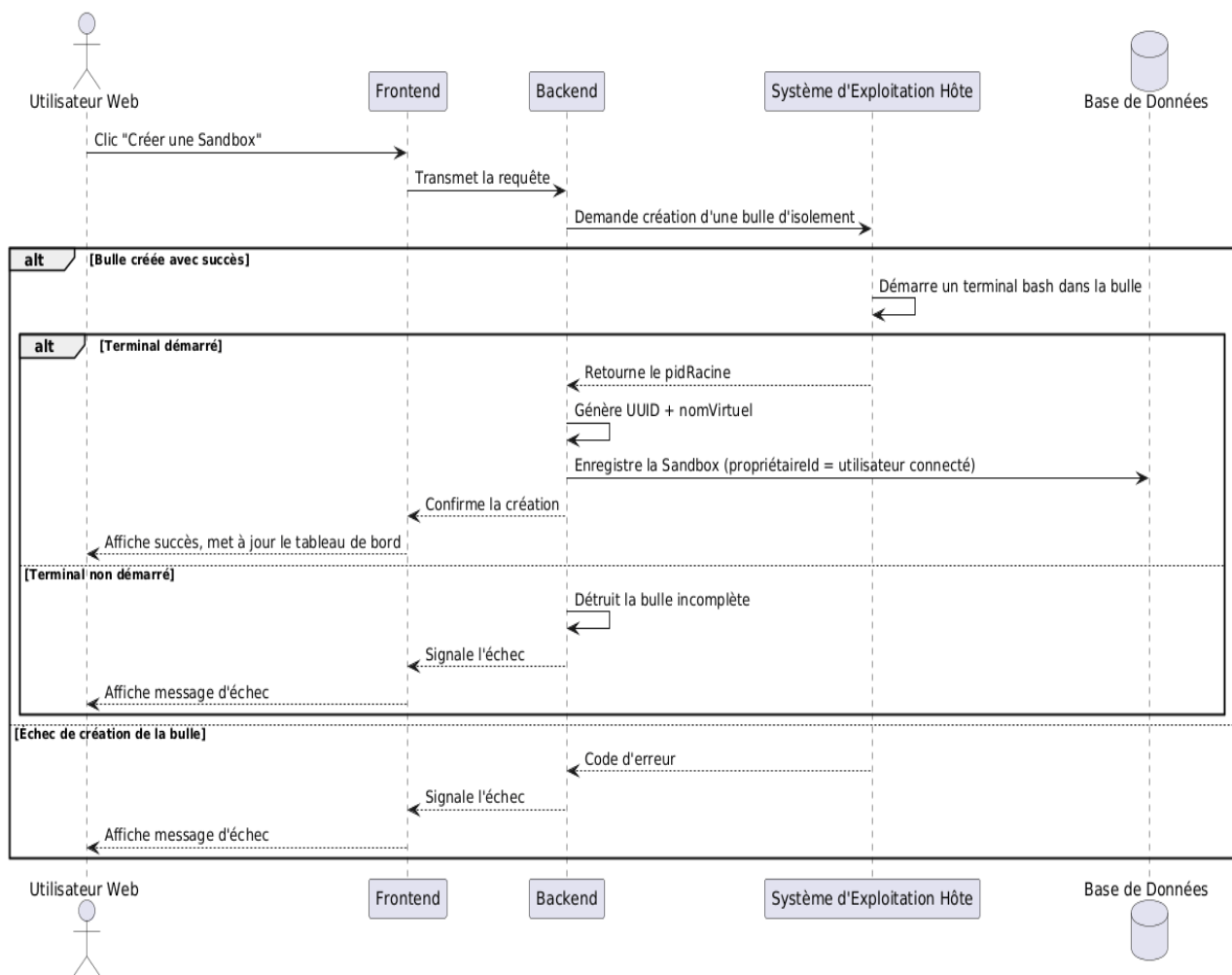


Figure 4 — DS1 Créer une Sandbox

4.3 DS2 — Lister les sandboxes

Ce scénario, le plus simple des six, décrit l'affichage du tableau de bord d'un utilisateur, avec ou sans sandbox existante.

Participants : Utilisateur Web · Frontend · Backend · Base de Données

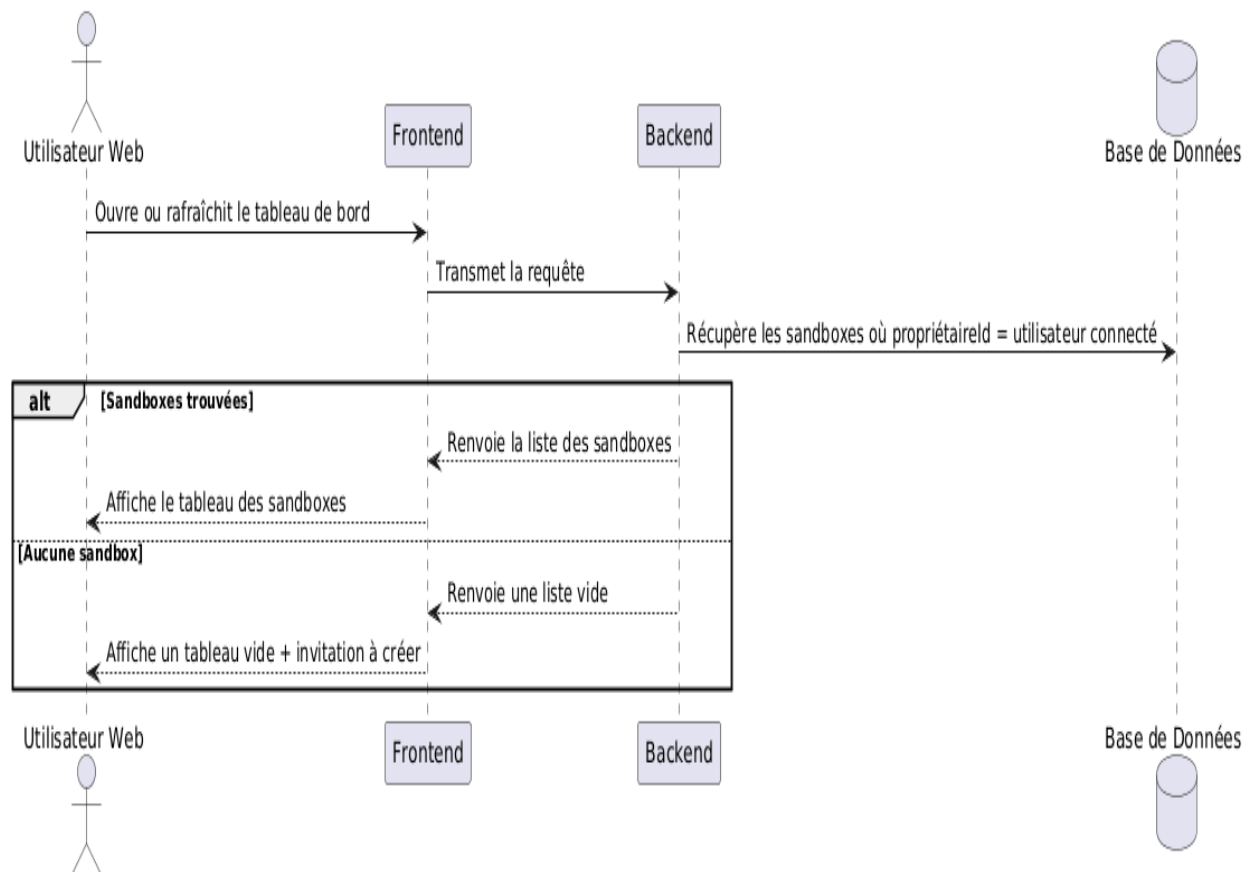


Figure 5 — DS2 Lister les sandboxes

4.4 DS3 — Exécuter une commande

Ce scénario décrit l'exécution d'une instruction saisie par l'utilisateur dans le terminal web, avec vérification préalable de l'autorisation et gestion des erreurs d'exécution.

Participants : Utilisateur Web · Frontend · Backend · Système d'Exploitation Hôte · Base de Données

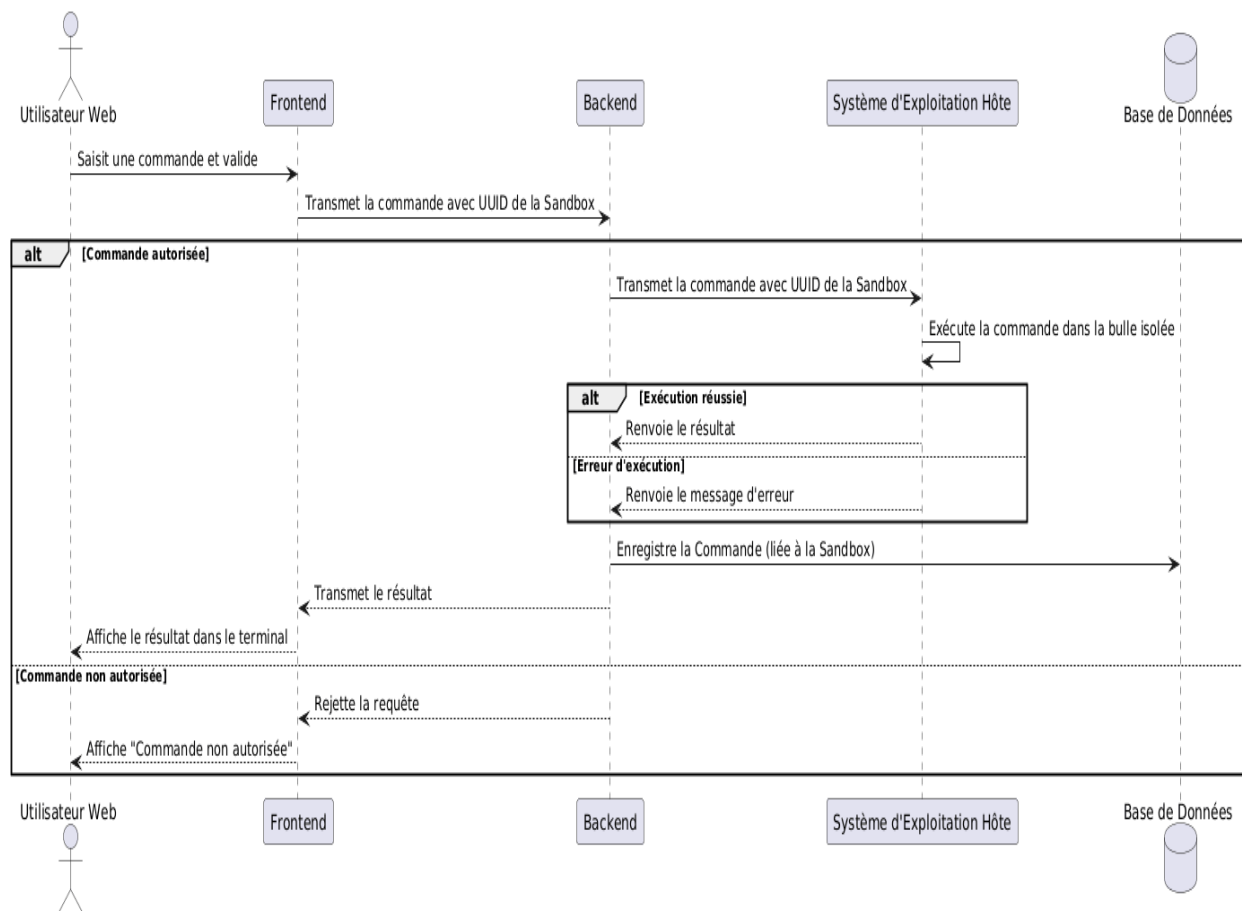


Figure 6 — DS3 Exécuter une commande

4.5 DS4 — Rappeler une commande précédente

Ce scénario décrit la navigation dans l'historique des commandes précédemment exécutées dans une sandbox, via les flèches Haut/Bas du terminal — un comportement similaire à celui d'un terminal Unix classique.

Participants : Utilisateur Web · Frontend · Backend · Base de Données

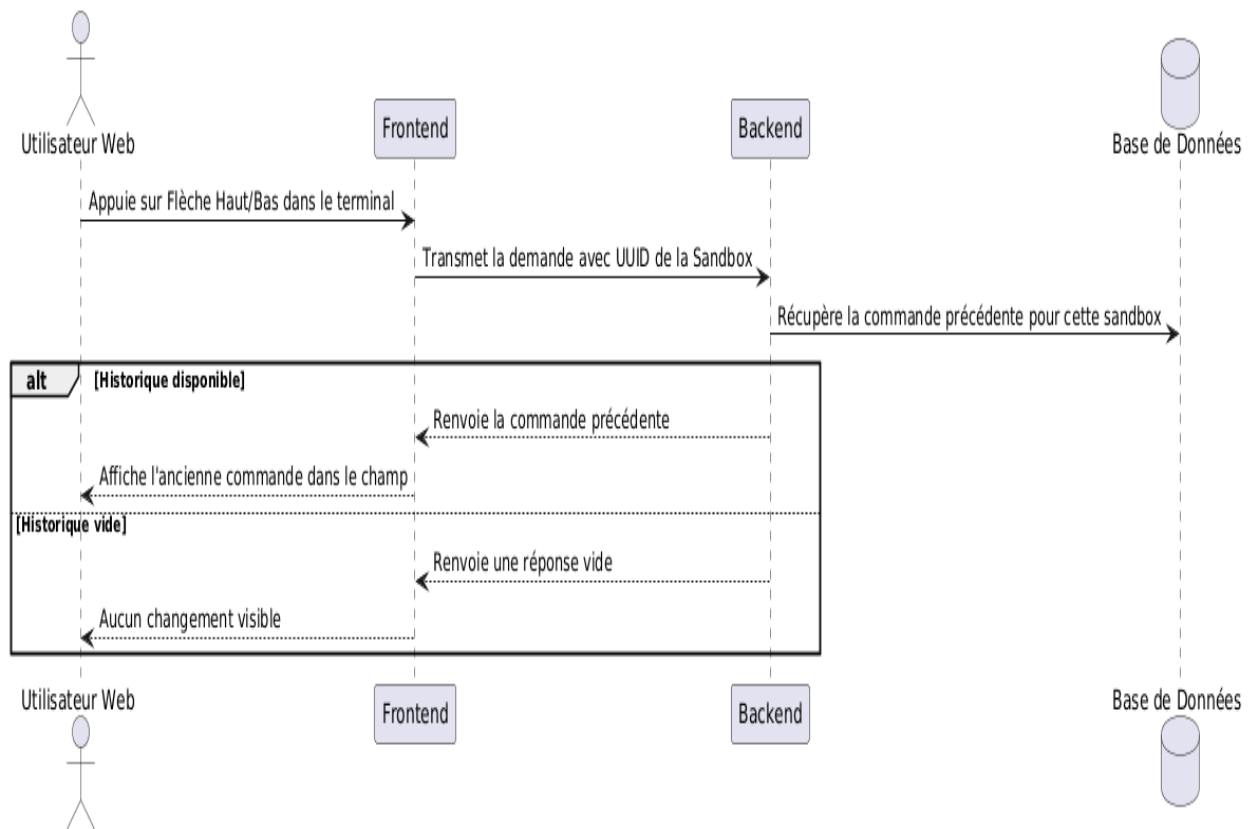


Figure 7 — DS4 Rappeler une commande précédente

4.6 DS5 — Supprimer une Sandbox

Ce scénario décrit la suppression d'une sandbox à la demande de l'utilisateur, avec libération des ressources système si un terminal racine est encore actif.

Participants : Utilisateur Web · Frontend · Backend · Système d'Exploitation Hôte · Base de Données

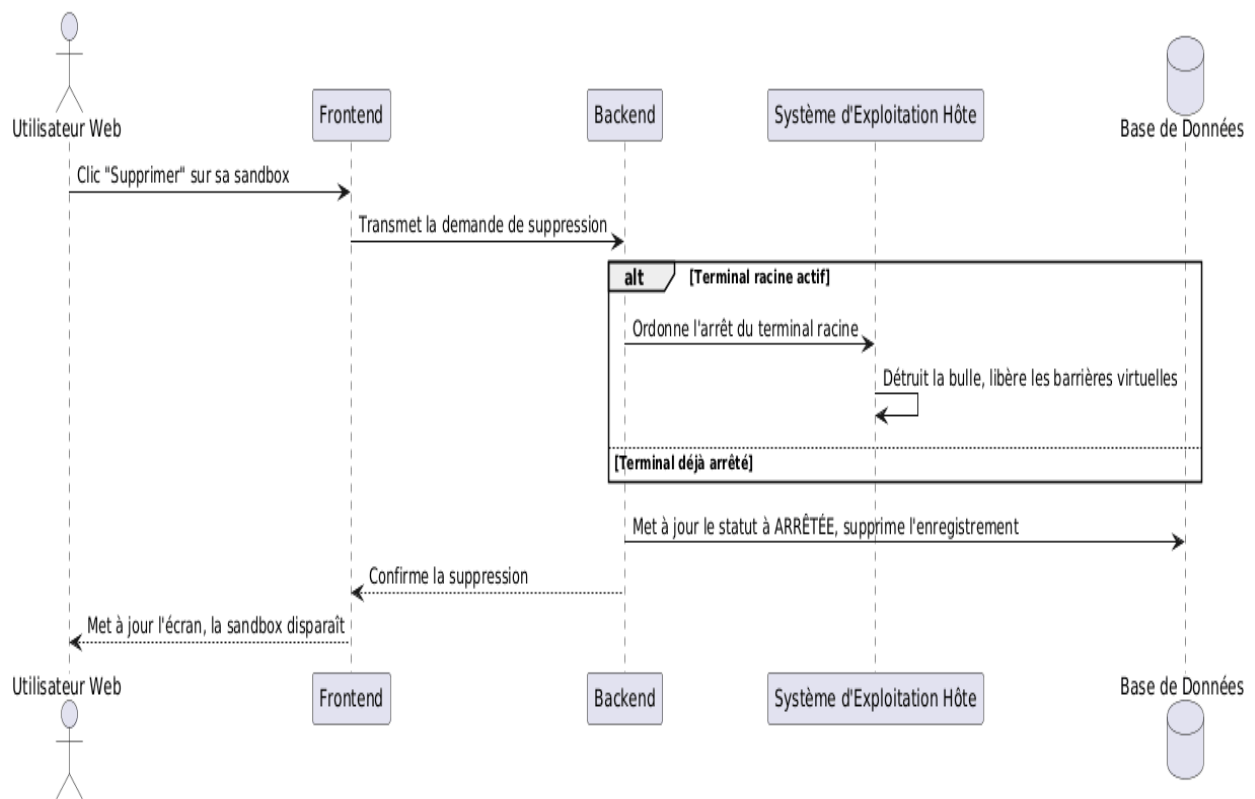


Figure 8 — DS5 Supprimer une Sandbox

5. SYNTHESE ET PROCHAINES ETAPES

5.1 Ce que la conception a permis de valider

- Une architecture 3-tier validée, avec une couche données dédoublée entre persistance (Base de Données) et exécution réelle (Système d'Exploitation Hôte) ;
- Une structure statique claire autour de trois classes : Utilisateur, Sandbox et Commande ;
- Un mécanisme d'isolation multi-utilisateur reposant sur le filtrage systématique par `proprietaireId` ;
- Une exécution du Backend sous un utilisateur Linux dédié, non-root ;
- Six diagrammes de séquence couvrant l'ensemble des cas d'usage : DS0 (authentification), DS1 (création), DS2 à DS5 (gestion courante et sécurité) ;
- Une séparation nette entre Frontend et Backend, sans logique métier côté client.

5.2 Prochaines étapes

- Implémentation avec React.js et Python Flask ;
- Développement de l'interface web, avec un terminal interactif basé sur WebSocket ;
- Déploiement sur un serveur Linux dédié ;
- Tests de sécurité selon les modules ATTACK et DEFENSE.